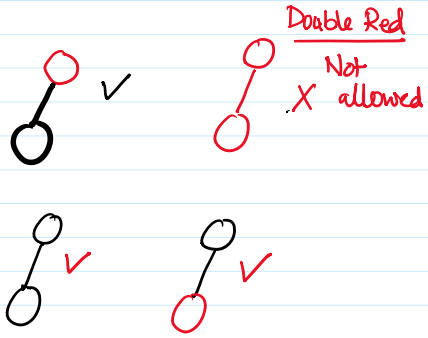
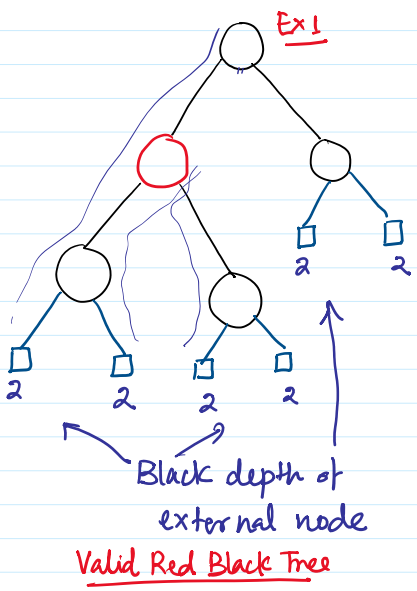
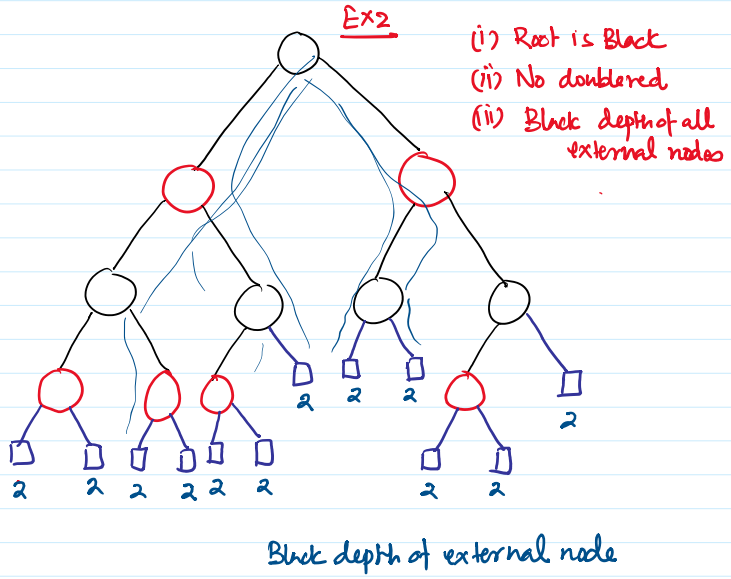
7) | Lecture 12-14 | Red Black Tree |

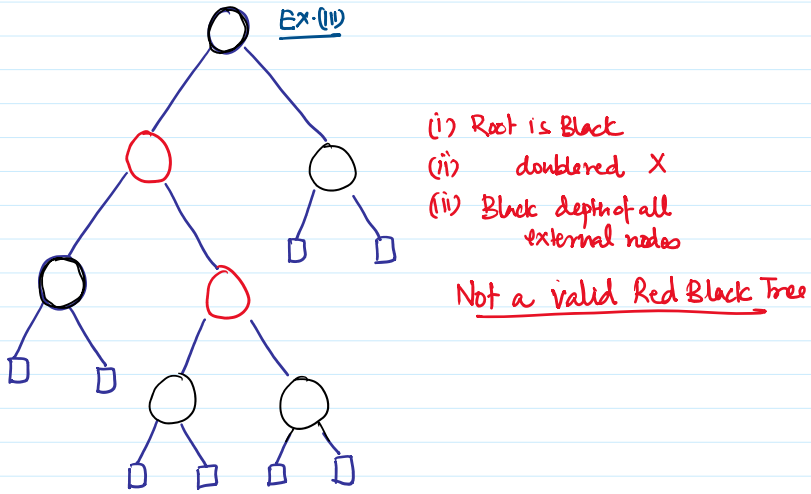
28 August 2026 14:51

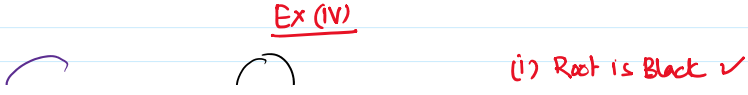
- A red black tree is a BST in which each node is either colored red or black
- Root is always colored black
- Red node can have only black children
- If a node of BST doesn't have left or right child then we add an external node
- External nodes are not colored
- The black depth of an external node is the number of black ancestors it has
- In a red black tree every external node has same black depth ✓



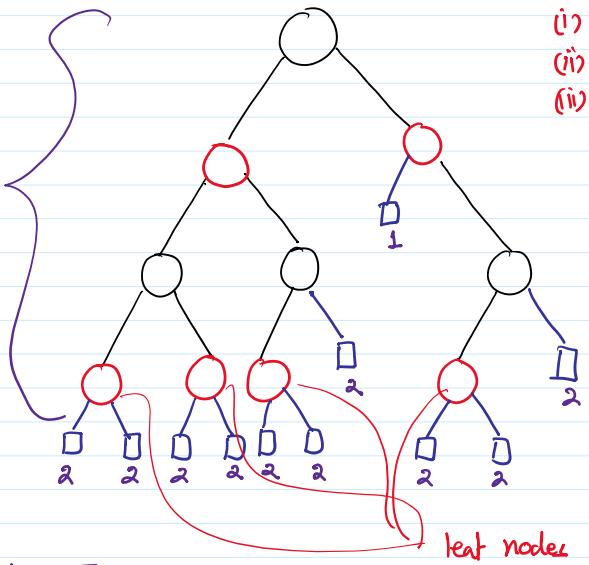








Ex (IV)



(i) Root is Black ✓
(ii) No doubletred ✓
(iii) Black depth of all external nodes ✗

Not a valid Red Black Tree

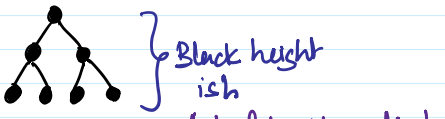
Height of Red Black Tree

n # of nodes in Red black tree T

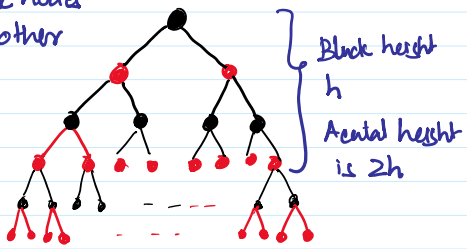
h is black height of T

$$2^h - 1 \leq n \leq 2^{2h} - 1$$

When there is no red node in the RB tree



When red and black nodes alternate with each other



$$\Rightarrow O(\log_4 n) \leq h \leq O(\log_2 n)$$

$$h = \theta(\log n)$$

(i) Search - Easy like Binary Search $\theta(\log n)$

(ii) Insertion

(iii) Deletion

Insertion in Red Black Tree

Input : Valid RB tree T, key k be the key that need to be inserted

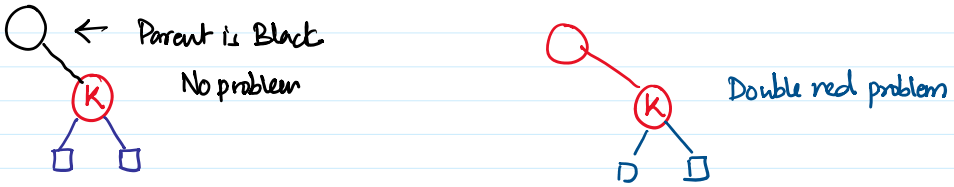
Output Valid RB Tree $T \cup \{k\}$

Step 1 Find appropriate position for k in T and add it at leaf level

Step 2 Color the new node with Red color.

Since newly inserted is colored red, the black depth of all external node remain same

However if parent of inserted node is red, then we have a double red problem

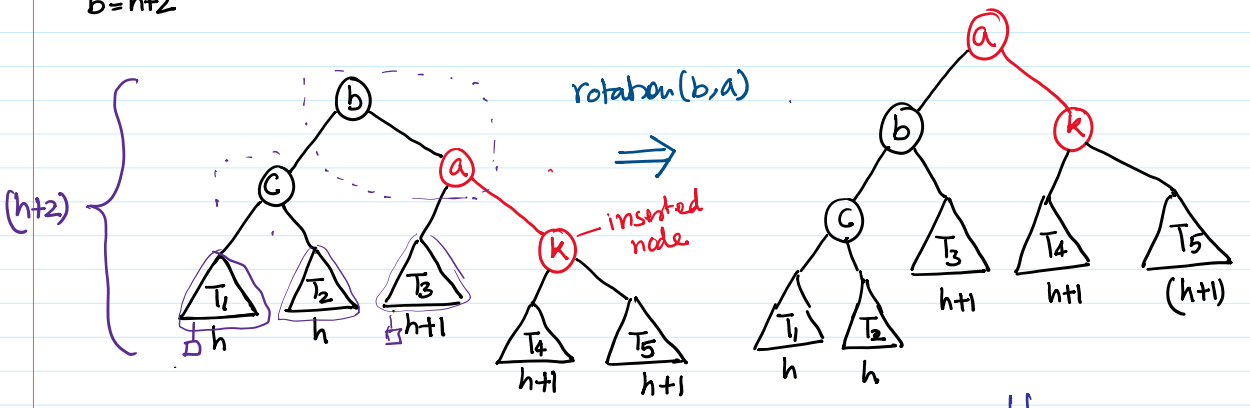


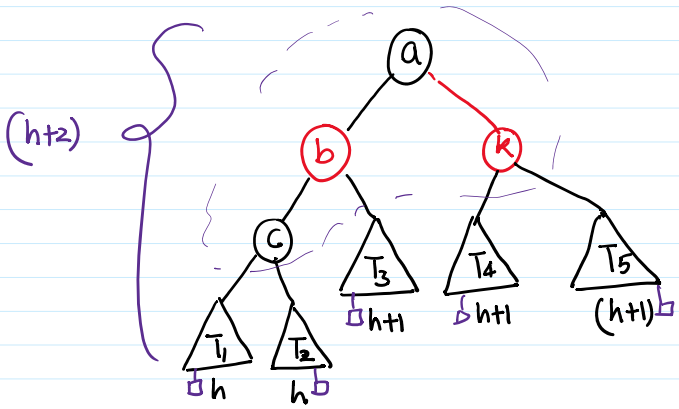
Case (i) Parent of inserted node (k) (a) is red. Parent of (a) must be black.

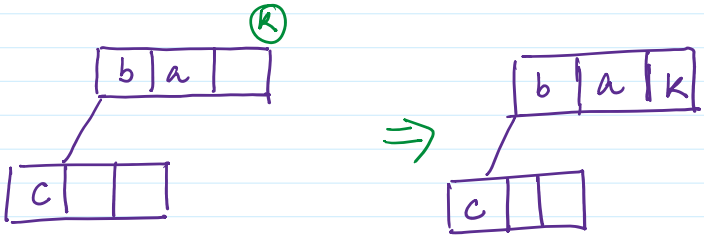
Let the child of (b) (say (c)) is black

(i) rotation
(ii) coloring

Original height of tree rooted $b = h+2$

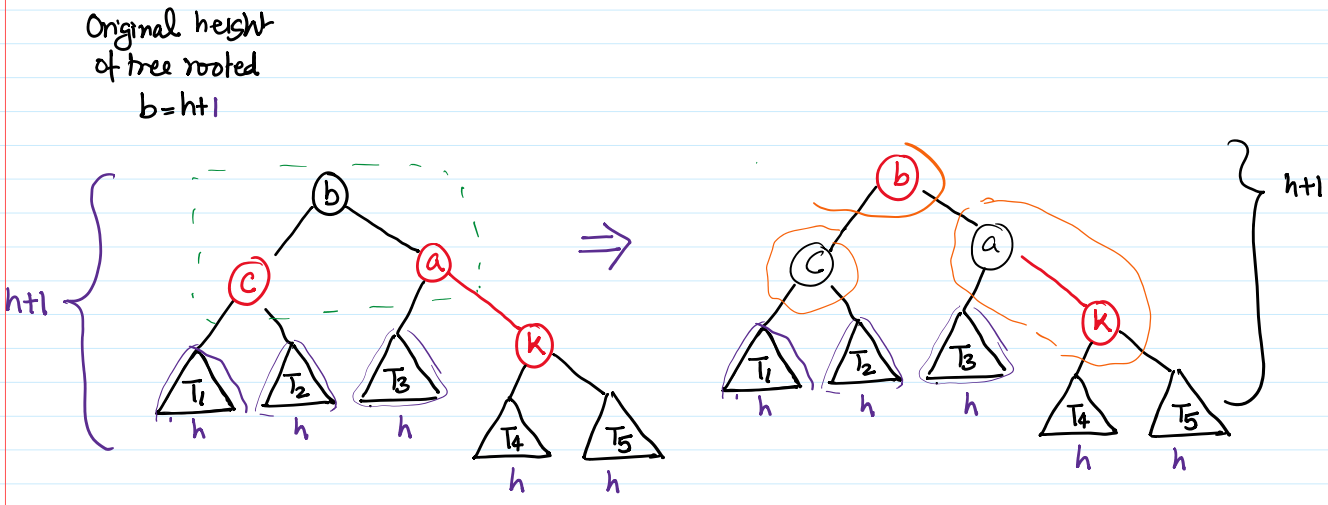




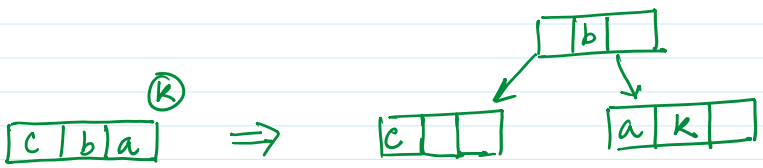


Case (ii) _ Parent of inserted node (k) (a) is red. Parent of (a) must be black.

Let the child of (b) (say (c)) is Red



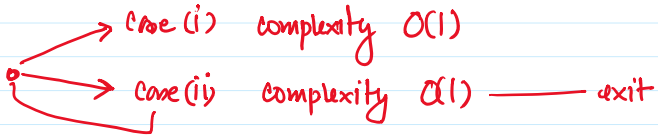
- The parrent of (b) can also be red. In that case double red problem moves upa level
  - We recurse the same algo at next level.
  - Eventually we might have to color root node red (if we keep on falling case(i) again & again)
  - In that case we recolor root as black. This increase the black height of overall tree by 1



Time Complexity

(i) Search & insertion like BST O(log n)

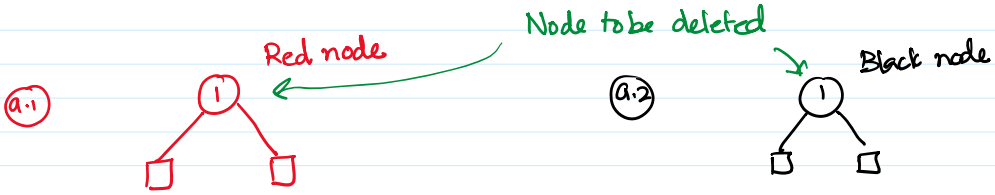
(i) Search & insertion like BST O(log n)



Deletion in Red Black Tree To delete a node in RedBlack tree we proceed like deleting a node in BST.

In BST the node that are going to be deleted is either a leaf node or parent to a leaf node.

Case (a) The node which is going to be deleted is leaf node

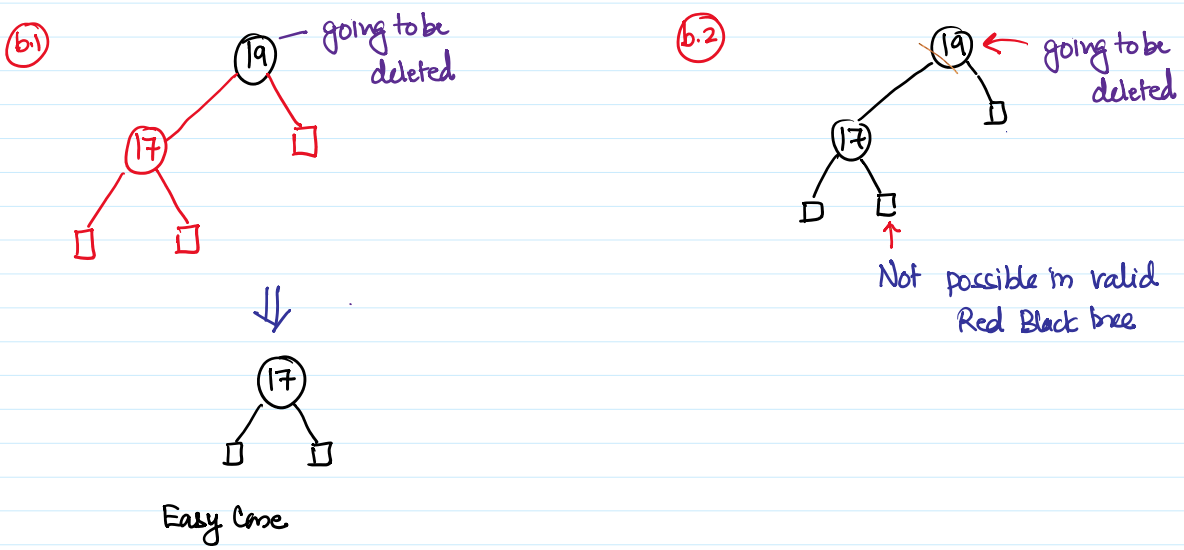


Can be deleted

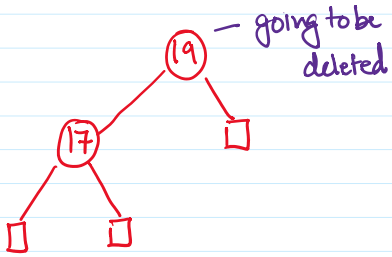
Easy Case

- We don't introduce double red problem
- Black depth of all external nodes should be same

Case (b) The node which is going to be deleted is parent to a leaf node

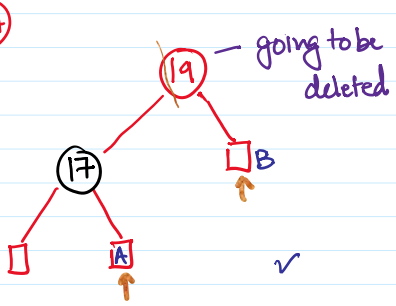


6.3



Not possible due to double red problem

6.4



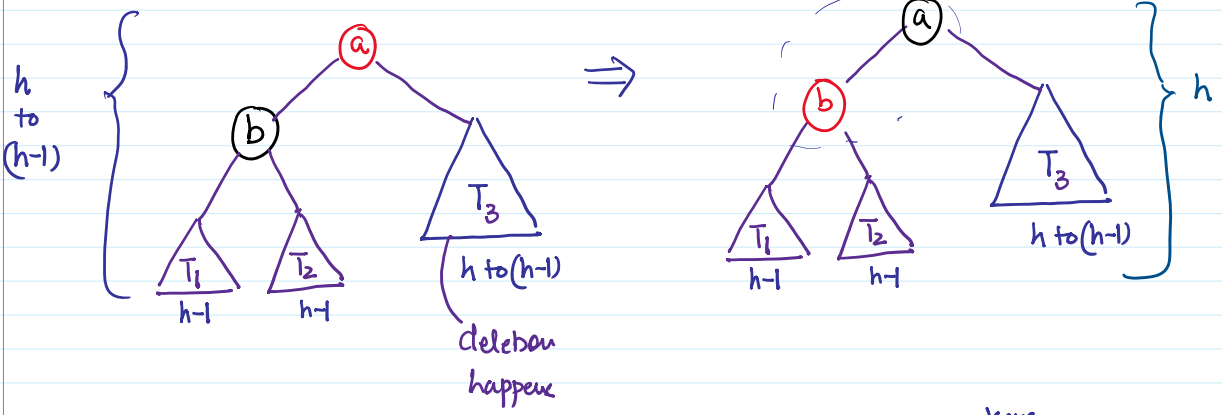
Not possible in valid Red Black tree as black depth of node A and B are not same

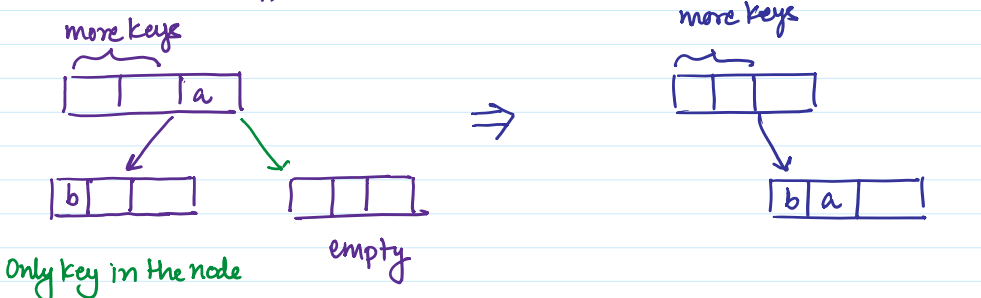
- W.l.o.g. we can assume that node that we are going to delete is black leaf node
- Removing this node will reduce black depth of some external node by 1
- Therefore in general step, we consider the problem of reorganising a subtree whose black depth reduced from h to (h-1).

Deletion Case 1.1 Deletion happens in T_3. let parent of T_3 is red node (a)

(a) has other child (b) of black color

let (b) doesn't have any red child.





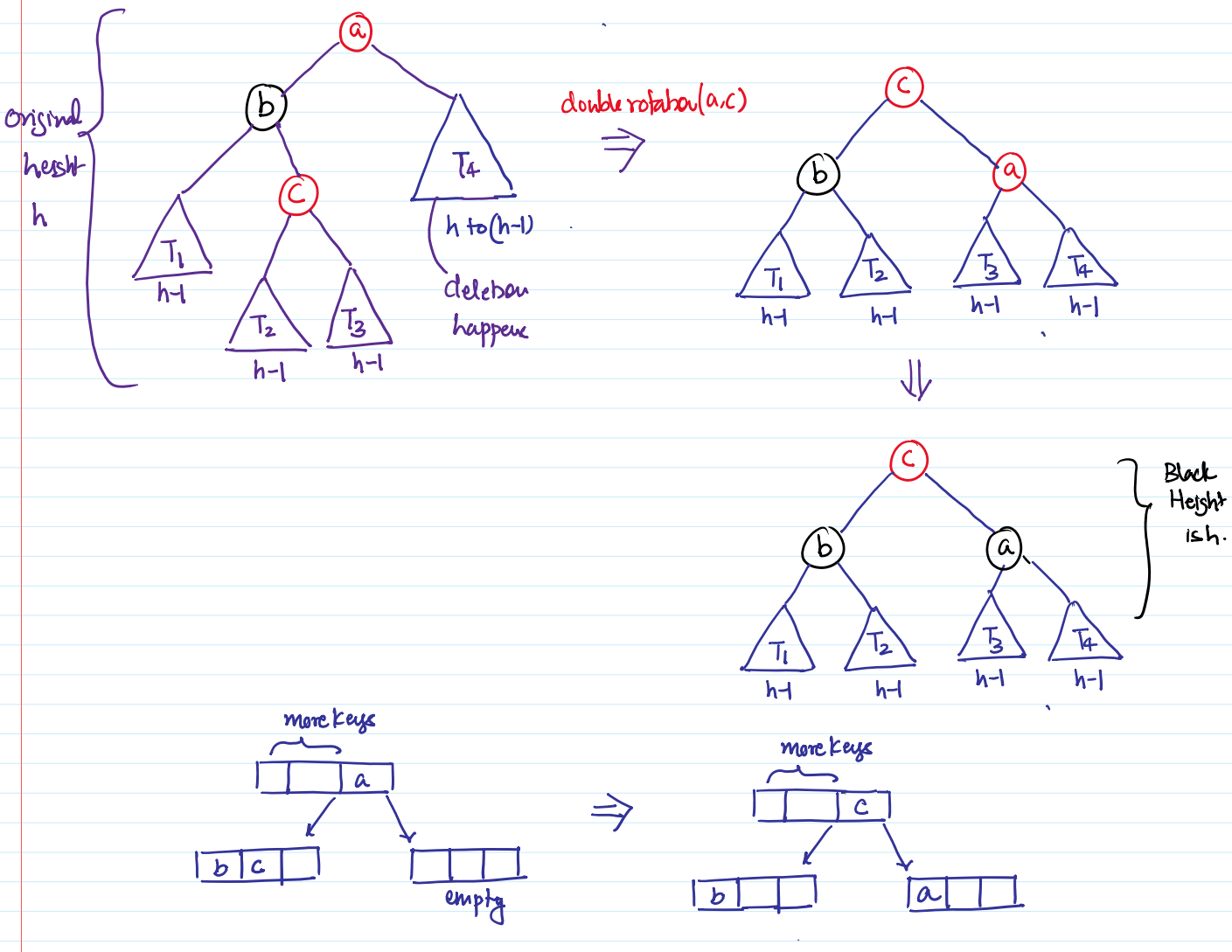
Only key in the node as b doesn't have any red child

empty

Case (1.2) Deletion happens in T_4 let parent of T_4 is red node (a)

(a) has other child (b) of black color

let (b) has red child (c)



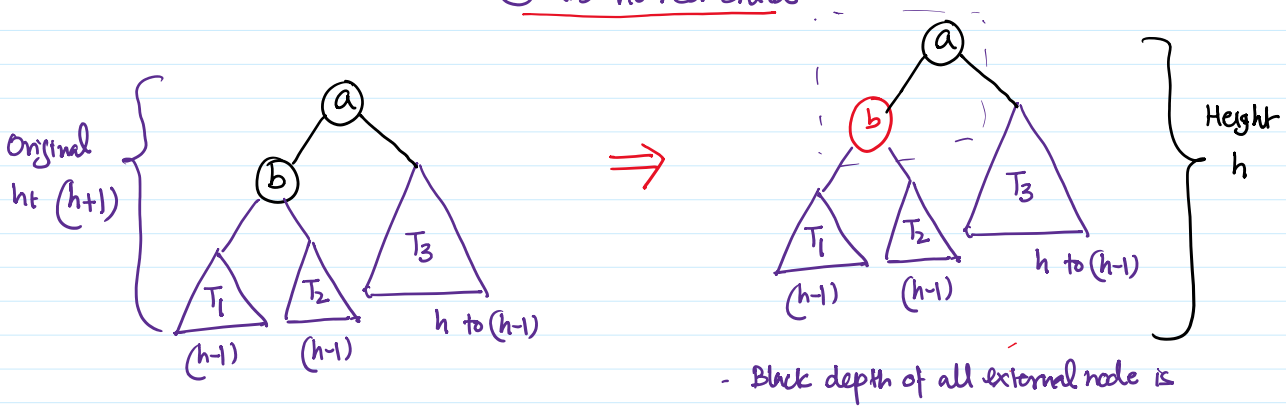
Case 2.1.1 Deletion happens in T_3 let parent of T_3 is black node (a)

(a) has other child (b) of black color

let (b) has no red child

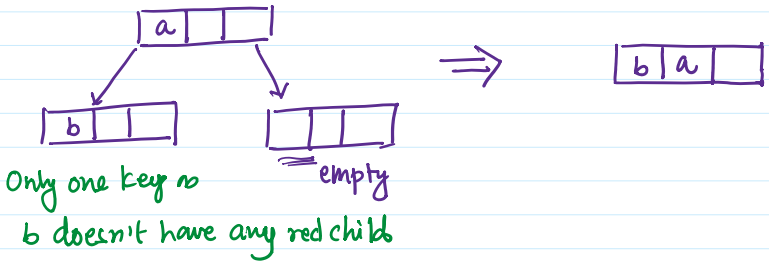
ⓐ has other child ⓑ of black color

let ⓑ has no red child



- Black depth of all external node is same now.
- Height of subtree rooted ⓐ reduced by 1

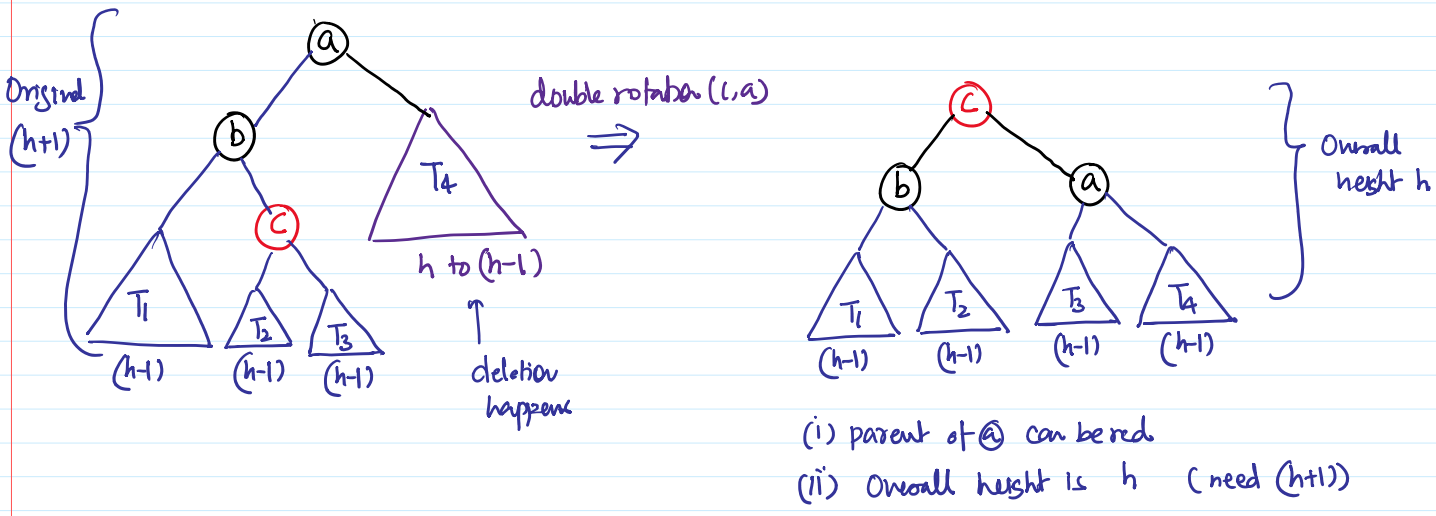
⇒ Recurse subtree rooted ⓐ as new T_3



Case 2.1.2 Deletion happens in T_4 let parent of T_4 is black node ⓐ

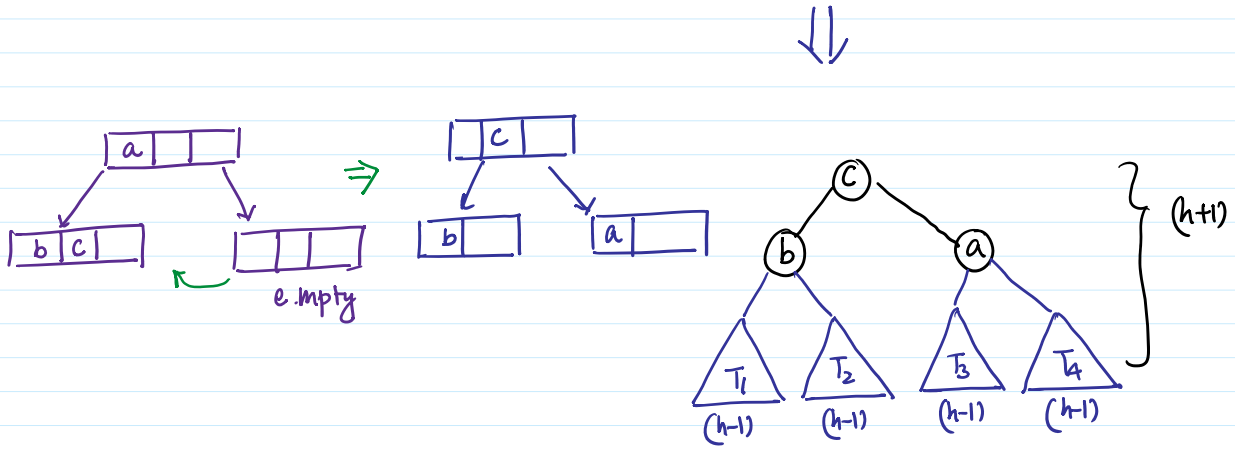
ⓐ has other child ⓑ of black color

let ⓑ has a red child say ⓒ



(i) parent of ⓐ can be red

(ii) Overall height is h (need (h+1))

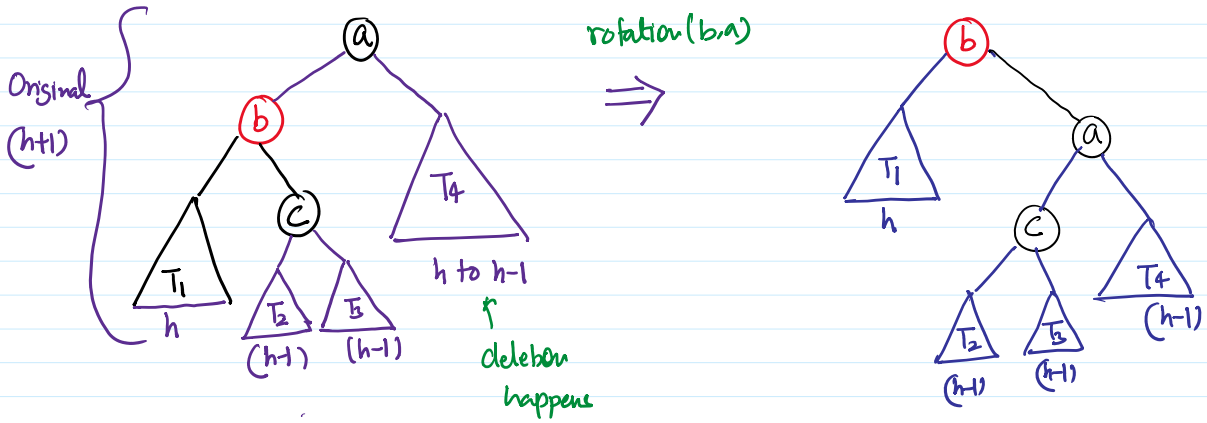


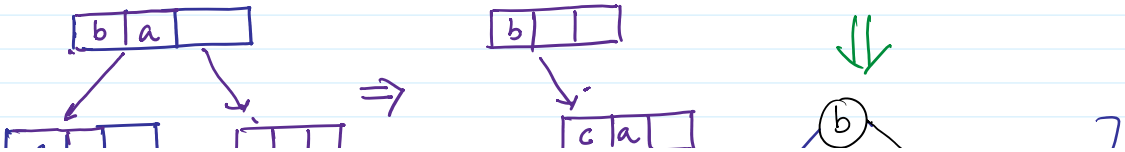
Case 2.2.1 Deletion happens in T_4 let parent of T_4 is black node (a)

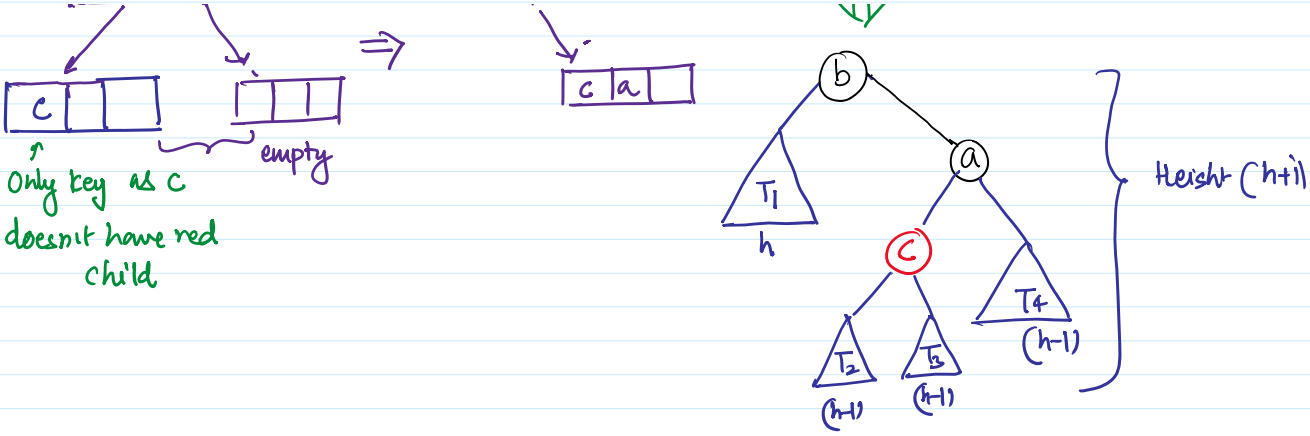
let (a) has other child (b) of red color

(b) has black child say (c)

let (c) doesn't have any red child





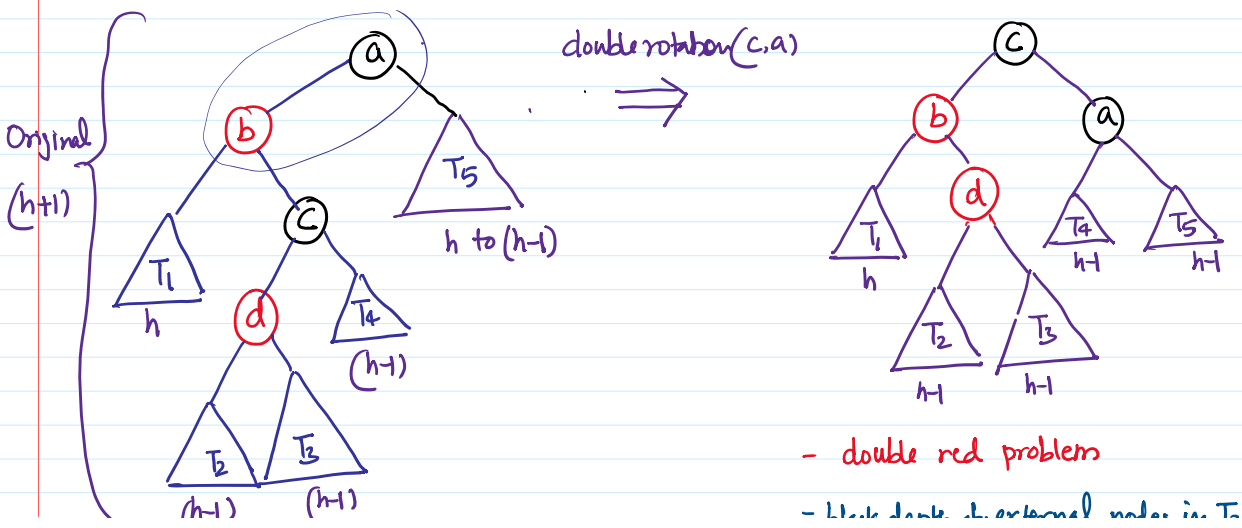


Case 2.2.2 Deletion happens in T_5 let parent of T_5 is black node (a)

let (a) has other child (b) of red color

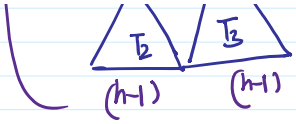
(b) has black child say (c)

let (c) has a red child (d)

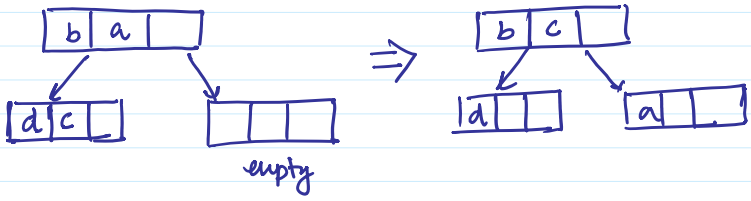


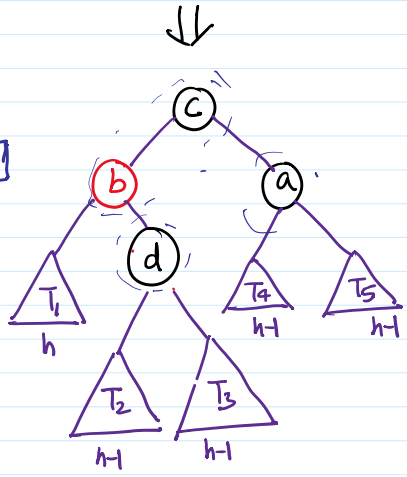
- double red problem

- black depth of external nodes in T_3, T_5



- double red problem
- black depth of external nodes in T_2, T_3 are h but remaining (h+1)





Algo. pseudocode

Theorem — why algo is correct

Asymptotic time / space complexity analysis

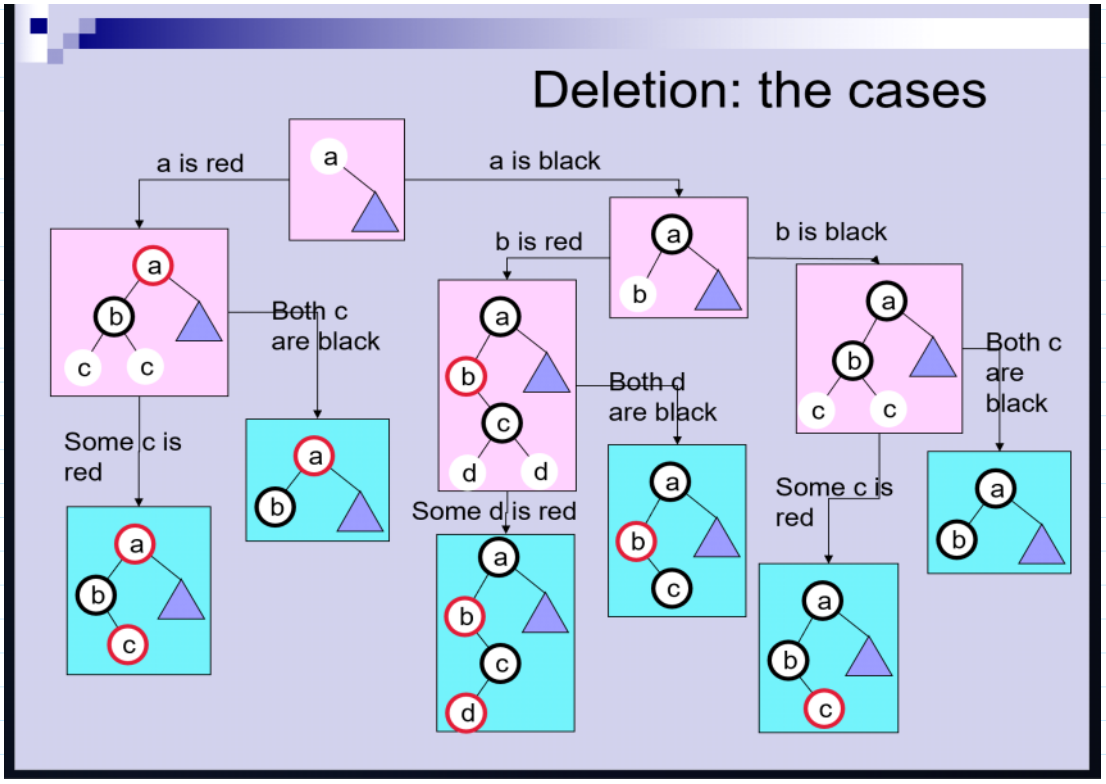
Input: RB tree, one node

Output

Time complexity

- Double red problem doesn't get introduced in deletion.
- Deletion in BST O(logn) time (Height is O(logn))
- If we keep on falling in case 2.1.1, then we need to go until root, which is O(logn) complexity
- Time complexity of rotation and recoloring O(1)

Overall time complexity O(logn)


Deletion: the cases
a is red
a is black
b is red
b is black
Both c are black
Both d are black
Both c are black
Some c is red
Some d is red
Some c is red